

01 — Structs

Definição e uso

```
struct Point {  
    float x;  
    float y;  
    float z;  
};  
  
Point p;  
p.x = 1.0f;  
p.y = 2.0f;  
p.z = 3.0f;  
  
Point q = {1.0f, 2.0f, 3.0f};    // inicialização agregada  
Point r{};                      // tudo zerado
```

Um struct é um **layout**: uma descrição de como vários valores são empacotados numa única região de memória. Nesta fase é só isso — sem construtores, sem métodos, sem herança. Que por acaso é exatamente o que um struct é em código de kernel.

Layout na memória

```
struct Point { float x, y, z; };    // 12 bytes
```

offset	campo
0	x (4 bytes)
4	y (4 bytes)
8	z (4 bytes)
total: 12	

Os campos ficam na ordem de declaração. Essa ordem é garantida, e é por isso que reordenar campos muda o `sizeof` — assunto do próximo arquivo.

Através de pointers

```
Point p{1,2,3};  
Point* pp = &p;  
  
pp->x;        // atalho  
(*pp).x;     // idêntico, e mais claro sobre o que está acontecendo
```

-> é `(*ptr)`. — dereference, depois acesso. Nada mais que isso.

Arrays de structs

```
Point points[100];           // 100 * sizeof(Point) bytes contíguos
points[5].x = 1.0f;
```

Contíguo, como qualquer array. Os elementos são structs em vez de escalares, e o stride é `sizeof(Point)` em vez de `sizeof(float)`.

```
Point points[3]           -- Array of Structures

offset:  0    4    8   12   16   20   24   28   32
         +---+---+---+---+---+---+---+---+
         | x0 | y0 | z0 | x1 | y1 | z1 | x2 | y2 | z2 |
         +---+---+---+---+---+---+---+---+
         \___ point 0 ___/\___ point 1 ___/
```

Ler todo x significa tocar os offsets 0, 12, 24 -- stride 12, não 4.

Essa última linha é o assunto inteiro do arquivo de teoria 03.

Passando structs

```
void f(Point p);           // copia sizeof(Point) bytes
void f(const Point& p);    // sem cópia
void f(const Point* p);    // sem cópia, compatível com C
```

Para um `Point` de 12 bytes a cópia é barata. Para um struct grande, não é — e o raciocínio do Módulo 05 se aplica sem mudança nenhuma.

Aninhamento

```
struct Vertex {
    Point position;
    Point normal;
    float u, v;
};
```

Layout se compõe: campos em ordem, cada um com seu próprio alinhamento. As regras de padding do próximo arquivo se aplicam em todos os níveis.

Por que isso importa para CUDA

Structs aparecem o tempo todo em código de GPU — vetores, números complexos, pequenos pacotes de parâmetros, `float4`.

`float4` em particular não é só um tipo de conveniência. São 16 bytes, alinhados a 16 bytes, e ler um deles é **uma única instrução que move 16 bytes**. Loads vetorizados são uma otimização padrão de banda de memória, e funcionam por causa das garantias de layout deste módulo.

O diagrama de stride acima é o motivo: se as 32 leituras de um warp são contíguas ou não depende inteiramente de como o dado está organizado, e struct é como o dado é organizado.